

Objectives

- Understand the significance of image enhancement.
- Try to use pretrained network to improve accuracy.
- Understand the importance of data annotation.

1 Image Enhancement

Has anyone noticed that our training set accuracy has reached over 90%, but the accuracy on the test set is only over 70%?



Figure 1: Comparison of Evaluation Accuracy and Training Accuracy

Why does this phenomenon occur? This is a typical example of overfitting.

Overfitting is a phenomenon in machine learning where a model learns the training data too well, capturing noise and details that do not generalize to new, unseen data. As a result, while the model performs exceptionally well on the training set, its performance drops significantly on the test set or in real-world applications.

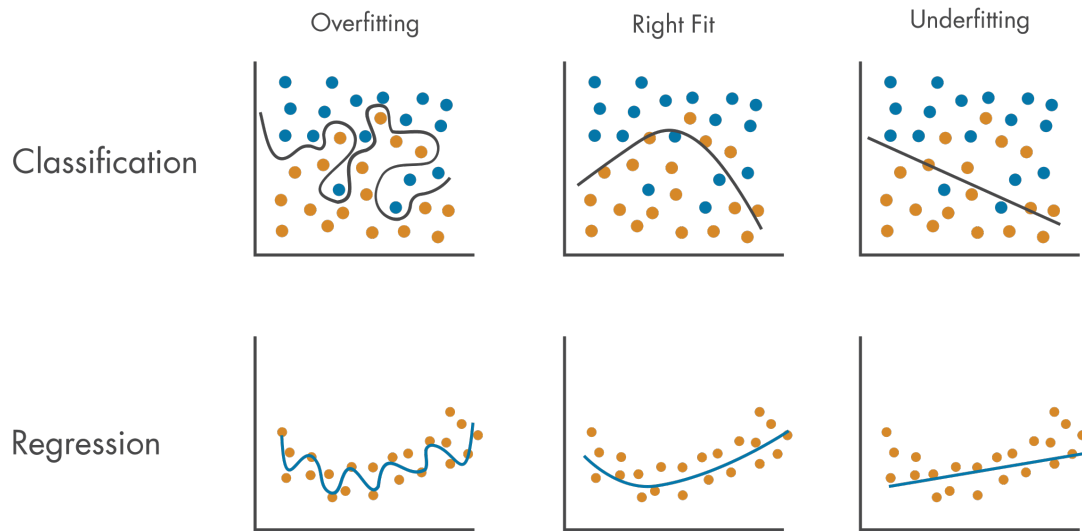


Figure 2: overfitting

Image enhancement plays a significant role in addressing overfitting in machine learning models, particularly in deep learning. There are several methods used for image enhancement, including:

1. Image Rescaling and Cropping: Resizing and cropping images can help focus on relevant features and improve the aspect ratio for better model performance.
2. Data Augmentation: Techniques such as rotation, flipping, and random cropping artificially expand the training dataset and help the model generalize better.
3. Contrast Stretching: This technique enhances the contrast between the darkest and brightest parts of an image, making features more distinguishable.
4. Deep Learning Approaches: Convolutional Neural Networks (CNNs) and Generative Adversarial Networks (GANs) can be utilized for advanced image enhancement, allowing for more sophisticated adjustments based on learned features.
5. ...

In this case, we can use image cropping and flipping operations to improve model generalization and reduce overfitting. In your jupyter file there are a section of code is specifically for data preprocessing, and a data augmentation module can be added here.

```
transform = {
    "training":
        transforms.Compose(
            [
                transforms.RandomHorizontalFlip(), #flip
                transforms.RandomRotation(10), # rotate 10 degrees.
                transforms.RandomCrop(44), # crop to 44x44
                transforms.Resize((48, 48)), # resize to 48x48
            ]
        )
}
```

```

transforms.ToTensor(),
transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)])),
    "evaluate":
transforms.Compose(
[
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)])),
}

```

We have differentiated the preprocessing for the training and inference processes; the inference process does not require image augmentation, so the corresponding code should change accordingly.

```

def load_data(path, batch_size, DataType):
    Dataset = ImageFolder(path, transform=transform[DataType])

#in loadTrain
trainset, train_loader = load_data(train_path, batch_size, "training")
#in loadTest
trainset, train_loader = load_data(train_path, batch_size, "evaluate")

```

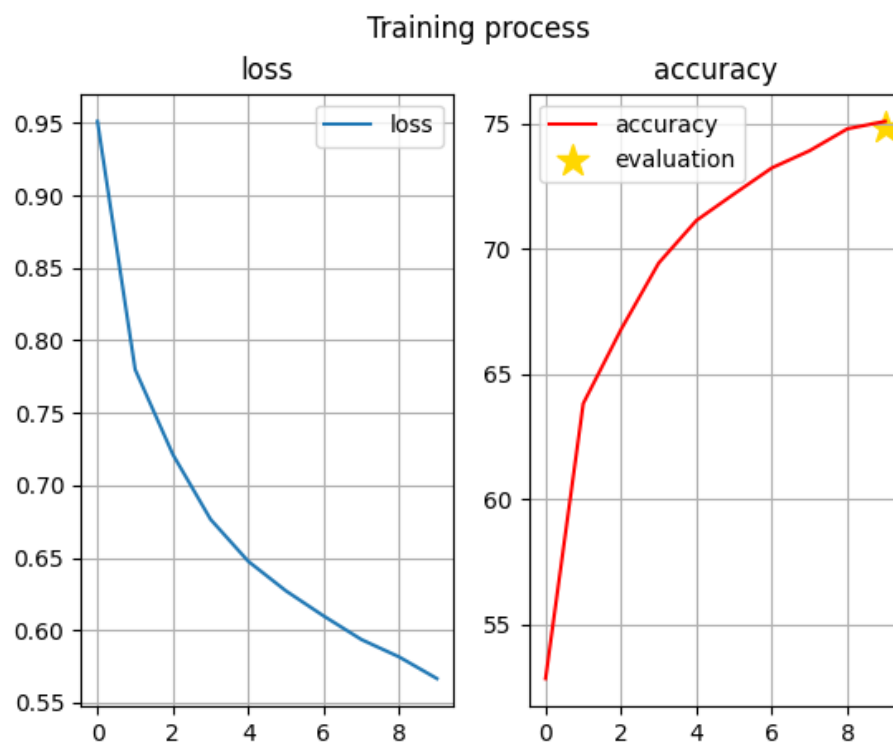


Figure 3: Comparison

After add image enhancement module, the evaluation accuracy improve two percents and it's very close to training process.

2 ResNet18 in Pytorch

ResNet, or Residual Network, is a type of deep learning architecture designed to facilitate the training of very deep neural networks. Introduced by Kaiming He and his colleagues in their 2015 paper, ResNet uses skip connections or shortcuts to jump over some layers. This approach helps mitigate the vanishing gradient problem that often occurs in deep networks, allowing for more effective training of architectures that can have hundreds or even thousands of layers.

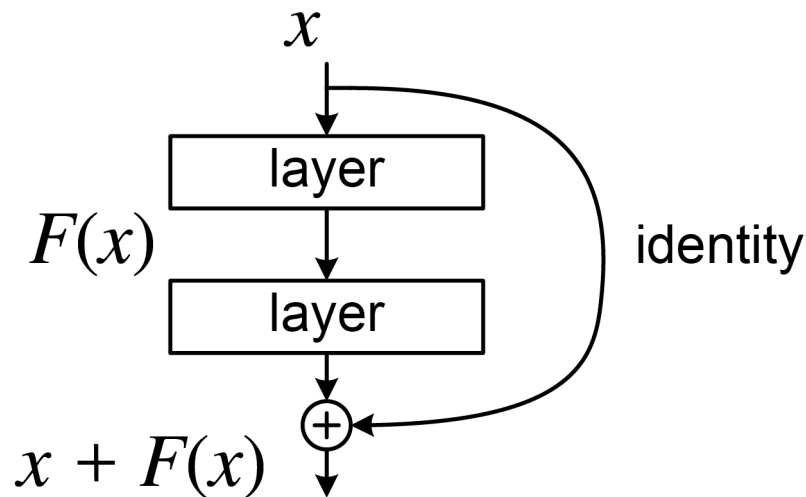


Figure 4: By LunarLullaby - Own work, CC BY-SA 4.0, <https://commons.wikimedia.org/w/index.php?curid=131458370>

The network architecture has been defined in pytorch. We can train our model based on a pre-trained model. Find *makeEmotionNet* function in your jupyter file and change it to the following format:

```
from torchvision import models
def makeEmotionNet(printtoggle=False):
    #model = emotionNet()
    model = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)
    num_features = model.fc.in_features
    model.fc = nn.Linear(num_features, 3)

    #loss function
    lossfun = nn.CrossEntropyLoss()
    #optimizer
    optimizer = torch.optim.Adam(model.parameters(), lr = .001, weight_decay=1e-5)

    return model, lossfun, optimizer
```

In my local tests, the accuracy was successfully improved to over 76%.

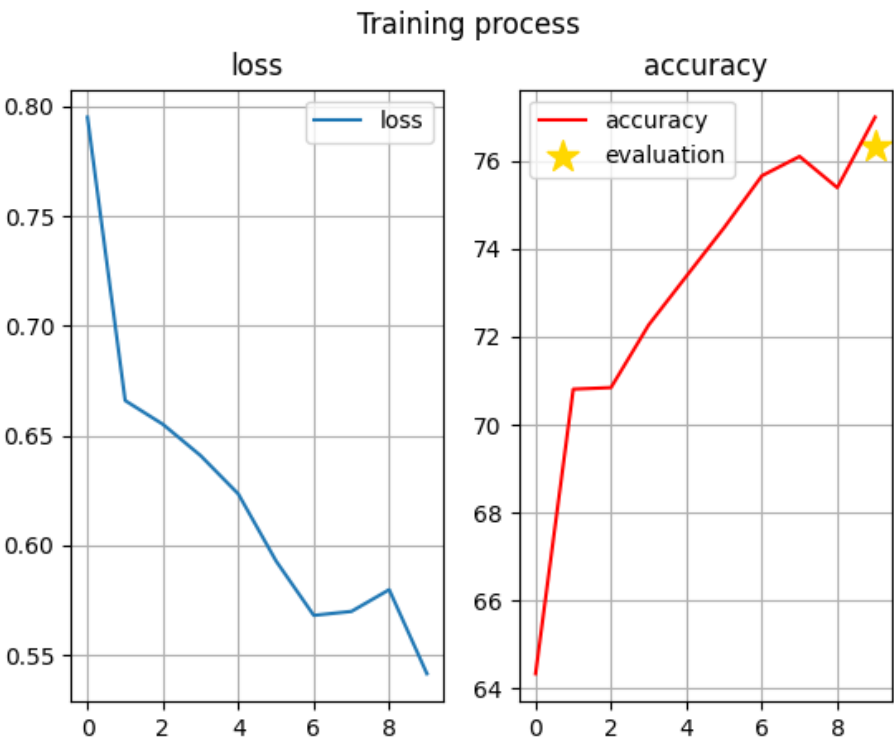


Figure 5: Comparison

We can use netron to observe the model structure. The residual connections can be clearly seen in the image.

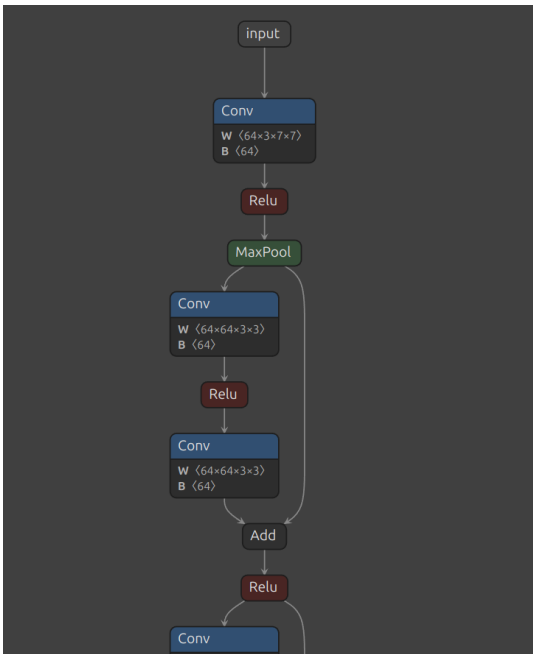


Figure 6: residual



Figure 7: Original Model

3 Data Annotation

In practical applications, if a model performs poorly, one of the simplest and most common solutions is to add more data. For instance, if an autonomous vehicle fails to recognize a car in certain road conditions, leading to a collision, the usual approach is to identify and collect data from those specific scenarios and add it to the dataset for training. This way, the model can better learn and recognize those road conditions.

In the sixth class, through the confusion matrix, it was observed that sadness and neutrality are easily confused. I recorded a video and found that for my old face, sadness and neutrality are often mixed up as well.

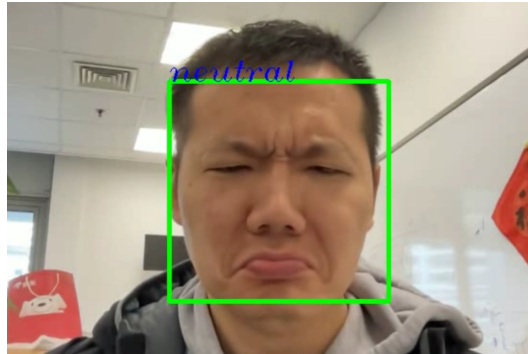


Figure 8: Incorrect classification

We can definitely avoid this situation by increasing the amount of data. Now there are many mature labeling tools like *LabelMe*, but in our case, we can simplify things. I recorded a video of sadness and then used OpenCV to extract all the detected expressions, saving them locally and directly placing them into the training data. The same approach applies to neutral expressions.

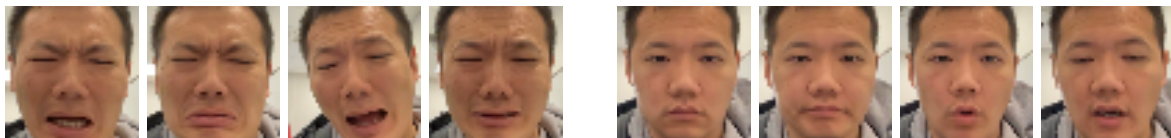


Figure 9: The four groups on the left are sad expressions, and the four groups on the right are neutral expressions

Move sad faces to *images/train/sad/* and move neutral faces to *images/train/neutral*.

After training, we successfully discovered that we finally classified the emotions correctly.

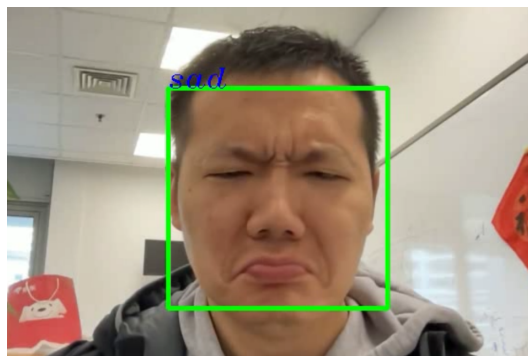


Figure 10: Correct classification

4 Test

Add data augmentation steps to your program and increase some training data, then train using ResNet18

5 Project report

After this class, you can start organizing all the project code and begin writing the project report.